

THE KARATSUBA MIDDLE PRODUCT FOR INTEGERS

DAVID HARVEY

ABSTRACT. We study the problem of computing middle products of multiple-precision integers. In particular we adapt the Karatsuba polynomial middle product algorithm to the integer case, showing how to efficiently mitigate the failure of bilinearity of the integer middle product noted by Hanrot, Quercia and Zimmermann. We discuss an implementation in GMP and applications to integer division.

1. INTRODUCTION

The aim of this paper is to study the problem of computing middle products of multiple-precision integers, by adapting existing algorithms for the polynomial middle product. Our key innovation is a practical solution to the problem of the *failure of bilinearity* of the integer middle product, which as pointed out in [4, p. 13] is the main obstruction to handling the integer case efficiently. In particular we describe a version of the Karatsuba middle product algorithm applicable to integers, based on the analogous algorithm for polynomials given in [5]. As an illustration of the utility of these techniques, we report on a careful implementation in GMP (the GNU Multiple Precision arithmetic library, [3]) of B -adic division, that is, computing a/b modulo B^k , where a and b are integers with k base- B digits, demonstrating a nontrivial speedup over competing algorithms.

We recall the definition and several properties of the polynomial middle product. Let $m \geq n$, and let $f = \sum_{i=0}^{m-1} f_i x^i$ and $g = \sum_{j=0}^{n-1} g_j x^j$ be polynomials in $R[x]$, where R is some coefficient ring (assumed commutative with identity). Their middle product is defined to be

$$(1) \quad \text{MP}_{m,n}(f, g) := \sum_{\substack{0 \leq i < m, 0 \leq j < n \\ n-1 \leq i+j \leq m-1}} f_i g_j x^{i+j-n+1}.$$

This operation corresponds to extracting the coefficients of x^k , for $n-1 \leq k \leq m-1$, from the usual product fg (see Figure 1). For simplicity we will consider mainly the *balanced* case, where $m = 2n - 1$; we denote this by $\text{MP}_n(f, g)$.

From a computational perspective, one of the most interesting properties of the polynomial middle product is its relation to the ordinary polynomial product via the ‘transposition principle’. In a suitably restricted model of computation, any algorithm for computing an ordinary $n \times n$ product may be ‘transposed’ to obtain an algorithm for computing $\text{MP}_n(f, g)$. Moreover, if time complexity is measured by the number of multiplications performed in R , the two algorithms have the *same* running time (the space complexity and number of additions and subtractions may increase by $O(n)$). For a precise statement, and details of the transformation, we refer the reader to [1].

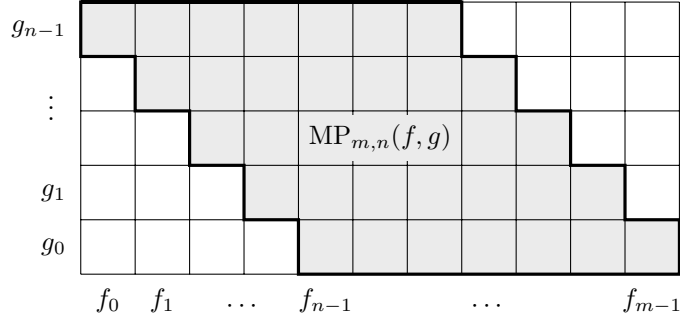


FIGURE 1. The middle product

For example, the classical multiplication algorithm performs n^2 coefficient multiplications. Its transpose, the classical middle product algorithm, simply multiplies out the n^2 term-by-term products appearing in (1) and accumulates appropriately. A more interesting example is the Karatsuba multiplication algorithm, which recursively splits the problem into three half-sized multiplications, thereby performing $n^{\log 3 / \log 2} \approx n^{1.58}$ coefficient multiplications (assuming that n is a power of two, and that Karatsuba is called recursively at every stage). Transposing it yields a Karatsuba middle product algorithm; one possible version is given in Algorithm 1 below, following [5, p. 417]. For simplicity we assume that $n = 2k \geq 2$ is even.

Algorithm 1: Karatsuba middle product for polynomials

Input: polynomials $f = \sum_{i=0}^{4k-2} f_i x^i$ and $g = \sum_{i=0}^{2k-1} g_i x^i$ in $R[x]$

Output: the middle product $\text{MP}_{2k}(f, g)$

$f^{(j)} \leftarrow \sum_{i=0}^{2k-2} f_{jk+i} x^i$ for $0 \leq j \leq 2$

$g^{(j)} \leftarrow \sum_{i=0}^{k-1} g_{jk+i} x^i$ for $0 \leq j \leq 1$

$P^{(0)} \leftarrow \text{MP}_k(f^{(0)} + f^{(1)}, g^{(1)})$

$P^{(1)} \leftarrow \text{MP}_k(f^{(1)}, g^{(0)} - g^{(1)})$

$P^{(2)} \leftarrow \text{MP}_k(f^{(1)} + f^{(2)}, g^{(0)})$

return $(P^{(0)} + P^{(1)}) + x^k(P^{(2)} - P^{(1)})$

The time complexity is $n^{1.58}$ multiplications in R , just as in the ordinary Karatsuba multiplication case.

To prove correctness of Algorithm 1, one could rely on the transposition principle and the correctness of the ordinary Karatsuba multiplication algorithm. Alternatively, one can give a direct proof using the bilinearity properties of the middle product: we have $\text{MP}(f + f', g) = \text{MP}(f, g) + \text{MP}(f', g)$ and $\text{MP}(f, g + g') = \text{MP}(f, g) + \text{MP}(f, g')$ for any polynomials f, f', g, g' , so the return value of Algorithm 1 is

$$\left(\text{MP}_k(f^{(0)}, g^{(1)}) + \text{MP}_k(f^{(1)}, g^{(0)}) \right) + x^k \left(\text{MP}_k(f^{(1)}, g^{(1)}) + \text{MP}_k(f^{(2)}, g^{(0)}) \right).$$

The decomposition of the middle product region in Figure 2 shows that this is equal to the desired result $\text{MP}_{2k}(f, g)$.

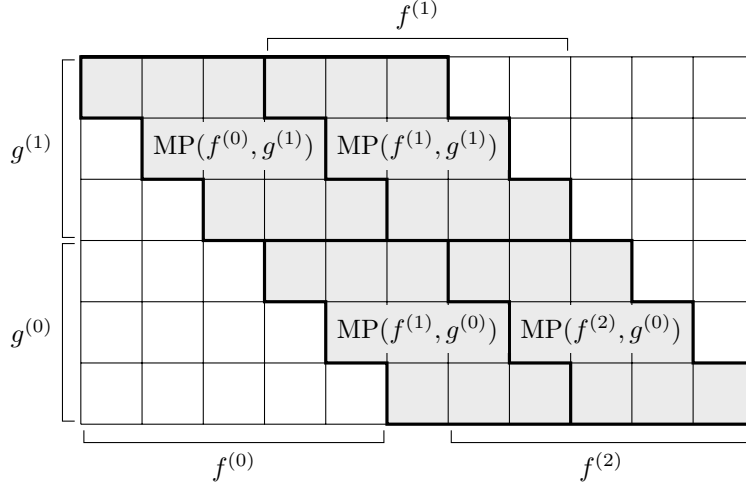


FIGURE 2. Karatsuba middle product for polynomials

We now consider the integer case. If $a = \sum_{i=0}^{m-1} a_i B^i$ and $b = \sum_{i=0}^{n-1} b_i B^i$ are two integers written in base B , the middle product of a and b is defined to be

$$(2) \quad \text{MP}_{m,n}(a, b) = \sum_{\substack{0 \leq i < m, 0 \leq j < n \\ n-1 \leq i+j \leq m-1}} a_i b_j B^{i+j-n+1}.$$

The integer case is more complicated than the polynomial case in at least two respects. First, $\text{MP}_{m,n}(a, b)$ will in general be more than $m - n + 1$ base- B digits long. We have $0 \leq \text{MP}_{m,n}(a, b) < nB^{m-n+2}$, so if we assume that $n < B$ (not a serious restriction in practice) then $\text{MP}_{m,n}(a, b) < B^{m-n+3}$, so $\text{MP}_{m,n}(a, b)$ is at most $m - n + 3$ digits long. Second, it is not quite true that $\text{MP}_{m,n}(a, b)$ corresponds to extracting the ‘middle digits’ of the ordinary product ab , because the sum fails to account for digits propagating upwards from products $a_i b_j$ with $i + j < n - 1$, including carries that could possibly affect all of the digits.

The main goal of this paper is to adapt the Karatsuba middle product algorithm (Algorithm 1) to the integer case. One is tempted to try to do this by replacing all arithmetic operations on polynomials by the corresponding operations for integers. Unfortunately, this approach fails because the middle product for integers is *not* bilinear. In general $\text{MP}(a + a', b) \neq \text{MP}(a, b) + \text{MP}(a', b)$, and similarly for linearity in the second variable. The problem is caused by internal carries: for example, a carry occurring in the addition $a + a'$ may contribute to $\text{MP}(a + a', b)$ but not to the sum $\text{MP}(a, b) + \text{MP}(a', b)$.

Our solution to this problem is to examine ‘error terms’ such as

$$\text{MP}_{m,n}(a + a' \bmod B^m, b) - (\text{MP}_{m,n}(a, b) + \text{MP}_{m,n}(a', b)),$$

which measures the failure of bilinearity of the integer middle product in the first variable. We give explicit expressions for such error terms, showing that they can be directly *computed* in linear time. Moreover this computation turns out to be quite cheap in practice. We then give an analogue of Algorithm 1 for the integer

case; the overall structure is the same, but the result is then corrected using suitable error terms.

2. THE KARATSUBA MIDDLE PRODUCT

We first give two lemmas that describe what error is introduced when trying to expand $\text{MP}(a \pm a', b)$ and $\text{MP}(a, b \pm b')$ using bilinearity.

For this we introduce some notation. Fix a representation base $B \geq 2$. We denote by $\langle a_0, \dots, a_{n-1} \rangle$ the integer $a = \sum_{i=0}^{n-1} a_i B^i$, where $0 \leq a_i < B$ for each i . For $\sigma \in \{+1, -1\}$, we define a function ADD_σ as follows. Let $u = \langle u_0, \dots, u_{n-1} \rangle$ and $v = \langle v_0, \dots, v_{n-1} \rangle$ be two integers, and let $\kappa_0 \in \{0, 1\}$. Put

$$\text{ADD}_\sigma(u, v, \kappa_0) := \langle w_0, \dots, w_{n-1} \rangle, (\kappa_1, \dots, \kappa_n),$$

where

$$u_i + \sigma(v_i + \kappa_i) = w_i + \sigma B \kappa_{i+1}, \quad 0 \leq w_i < B, \quad \kappa_i \in \{0, 1\},$$

for $0 \leq i < n$. That is, ADD_{+1} takes as input two n -digit integers u and v and an incoming carry κ_0 , and computes their sum w modulo B^n , the internal carries $\kappa_1, \dots, \kappa_{n-1}$, and the carry-out κ_n . Similarly, ADD_{-1} takes as input u and v and an incoming borrow κ_0 , and computes their difference w modulo B^n , the internal borrows $\kappa_1, \dots, \kappa_{n-1}$, and the borrow-out κ_n .

Lemma 1. *Let $m \geq n$, and let $u = \langle u_0, \dots, u_{m-1} \rangle$, $v = \langle v_0, \dots, v_{m-1} \rangle$ and $x = \langle x_0, \dots, x_{n-1} \rangle$ be integers. Let $\sigma \in \{+1, -1\}$ and $\kappa_0 \in \{0, 1\}$. Let $w, (\kappa_1, \dots, \kappa_m) = \text{ADD}_\sigma(u, v, \kappa_0)$. Then*

$$\begin{aligned} \text{MP}_{m,n}(w, x) &= \text{MP}_{m,n}(u, x) + \sigma \text{MP}_{m,n}(v, x) \\ &\quad + \sigma \left(\sum_{j=0}^{n-1} x_j \kappa_{n-1-j} - B^{m-n+1} \sum_{j=0}^{n-1} x_j \kappa_{m-j} \right). \end{aligned}$$

Proof. The error term $\text{MP}(w, x) - (\text{MP}(u, x) + \sigma \text{MP}(v, x))$ is equal to

$$\begin{aligned} &\sum_{j=0}^{n-1} \sum_{i=n-1-j}^{m-1-j} (w_i - u_i - \sigma v_i) x_j B^{i+j-n+1} \\ &= \sigma \sum_{j=0}^{n-1} x_j B^{j-n+1} \sum_{i=n-1-j}^{m-1-j} (\kappa_i - B \kappa_{i+1}) B^i \\ &= \sigma \sum_{j=0}^{n-1} x_j B^{j-n+1} (B^{n-1-j} \kappa_{n-1-j} - B^{m-j} \kappa_{m-j}), \end{aligned}$$

which is the desired result. $\square$

The key step in the above proof is the telescoping sum, which reduces the number of unwanted carry terms from $O((m-n)n)$ to only $O(n)$. Roughly speaking, all carries cancel out except those along the boundary of the middle product region. Both sums $\sum_{j=0}^{n-1} x_j \kappa_{n-1-j}$ and $\sum_{j=0}^{n-1} x_j \kappa_{m-j}$ are bounded by nB , so are at most two digits long if $n < B$. The first sum may be regarded as a correction to the low-order digits of $\text{MP}(w, x)$, and the second sum as a correction to its high-order digits.

Lemma 2. Let $m \geq n$, and let $x = \langle x_0, \dots, x_{m-1} \rangle$, $u = \langle u_0, \dots, u_{n-1} \rangle$ and $v = \langle v_0, \dots, v_{n-1} \rangle$ be integers. Let $\sigma \in \{+1, -1\}$ and $\kappa_0 \in \{0, 1\}$. Let $w, (\kappa_1, \dots, \kappa_n) = \text{ADD}_\sigma(u, v, \kappa_0)$. Then

$$\begin{aligned} \text{MP}_{m,n}(x, w) &= \text{MP}_{m,n}(x, u) + \sigma \text{MP}_{m,n}(x, v) \\ &+ \sigma \left(\sum_{i=0}^{n-1} x_i \kappa_{n-1-i} - B^{m-n+1} \sum_{i=m-n}^{m-1} x_i \kappa_{m-i} \right. \\ &\quad \left. + \kappa_0 \sum_{i=n}^{m-1} x_i B^{i-n+1} - \kappa_n \sum_{i=0}^{m-n-1} x_i B^{i+1} \right). \end{aligned}$$

Note that when we use Lemma 2 in the Karatsuba middle product algorithm below, we will arrange that $\kappa_0 = \kappa_n = 0$, so that the last two of the four terms above vanish.

Proof. The error term $\text{MP}(x, w) - (\text{MP}(x, u) + \sigma \text{MP}(x, v))$ is equal to

$$\sum_{i=0}^{m-1} \sum_{j=\alpha_i}^{\beta_i} x_i (w_j - u_j - \sigma v_j) B^{i+j-n+1},$$

where $\alpha_i = \max(0, n-1-i)$ and $\beta_i = \min(n-1, m-1-i)$. This is equal to

$$\sigma \sum_{i=0}^{m-1} x_i B^{i-n+1} \sum_{j=\alpha_i}^{\beta_i} (\kappa_j - B \kappa_{j+1}) B^j = \sigma \sum_{i=0}^{m-1} x_i B^{i-n+1} (B^{\alpha_i} \kappa_{\alpha_i} - B^{\beta_i+1} \kappa_{\beta_i+1}).$$

The terms involving α_i may be written as

$$\sigma \sum_{i=0}^{n-1} x_i B^{i-n+1} B^{n-1-i} \kappa_{n-1-i} + \sigma \sum_{i=n}^{m-1} x_i B^{i-n+1} B^0 \kappa_0,$$

and those involving β_i as

$$-\sigma \sum_{i=0}^{m-n-1} x_i B^{i-n+1} B^n \kappa_n - \sigma \sum_{i=m-n}^{m-1} x_i B^{i-n+1} B^{m-i} \kappa_{m-i},$$

accounting for the four sums appearing in the statement of the lemma. $\square$

We now describe a Karatsuba middle product algorithm (Algorithm 2), assuming for simplicity that $n = 2k \geq 2$ is even. We denote ADD_{+1} and ADD_{-1} by simply ADD and SUB .

Proposition 3. Algorithm 2 correctly computes the middle product $\text{MP}_{2k}(a, b)$.

Before giving the proof of correctness, we make some remarks on the structure of the algorithm and its relation to Algorithm 1. The ADD call corresponds to computing $f^{(0)} + f^{(1)}$ and $f^{(1)} + f^{(2)}$. If $\tau = 1$, the SUB call corresponds to computing $g^{(0)} - g^{(1)}$. If $\tau = -1$, the SUB call corresponds to a modified version of Algorithm 1 where $g^{(0)} - g^{(1)}$ and $P^{(1)}$ are replaced by $g^{(1)} - g^{(0)}$ and $-P^{(1)}$. The reason for introducing τ is to ensure that the SUB call does not generate a borrow, so that $c'_k = 0$, eliminating the last error term appearing in Lemma 2. Assuming uniformly distributed random input, this strategy saves $O(k)$ digit subtractions on average; the only additional cost is the comparison needed to determine τ , which costs $O(1)$ digit comparisons on average (or $O(k)$ comparisons in the worst case).

Algorithm 2: Karatsuba middle product for integers**Input:** integers $a = \langle a_0, \dots, a_{4k-2} \rangle$ and $b = \langle b_0, \dots, b_{2k-1} \rangle$ **Output:** the middle product $\text{MP}_{2k}(a, b)$ $\langle s_0, \dots, s_{3k-2} \rangle, \langle c_1, \dots, c_{3k-1} \rangle \leftarrow \text{ADD}(\langle a_0, \dots, a_{3k-2} \rangle, \langle a_k, \dots, a_{4k-2} \rangle, 0)$ **if** $\langle b_0, \dots, b_{k-1} \rangle \geq \langle b_k, \dots, b_{2k-1} \rangle$ **then** $\tau \leftarrow +1$ **else** $\tau \leftarrow -1$

$$\langle d_0, \dots, d_{k-1} \rangle, \langle c'_1, \dots, c'_k \rangle \leftarrow \begin{cases} \text{SUB}(\langle b_0, \dots, b_{k-1} \rangle, \langle b_k, \dots, b_{2k-1} \rangle, 0) & \tau = +1 \\ \text{SUB}(\langle b_k, \dots, b_{2k-1} \rangle, \langle b_0, \dots, b_{k-1} \rangle, 0) & \tau = -1 \end{cases}$$
 $P^{(0)} \leftarrow \text{MP}_k(\langle s_0, \dots, s_{2k-2} \rangle, \langle b_k, \dots, b_{2k-1} \rangle)$ $P^{(1)} \leftarrow \text{MP}_k(\langle a_k, \dots, a_{3k-2} \rangle, \langle d_0, \dots, d_{k-1} \rangle)$ $P^{(2)} \leftarrow \text{MP}_k(\langle s_k, \dots, s_{3k-2} \rangle, \langle b_0, \dots, b_{k-1} \rangle)$ $\tilde{P}^{(0)} \leftarrow P^{(0)} - \sum_{j=0}^{k-2} b_{k+j} c_{k-1-j} + B^k \sum_{j=0}^{k-1} b_{k+j} c_{2k-1-j}$ $\tilde{P}^{(1)} \leftarrow P^{(1)} + \sum_{j=0}^{k-2} a_{k+j} c'_{k-1-j} - B^k \sum_{j=0}^{k-2} a_{2k+j} c'_{k-1-j}$ $\tilde{P}^{(2)} \leftarrow P^{(2)} - \sum_{j=0}^{k-1} b_j c_{2k-1-j} + B^k \sum_{j=0}^{k-1} b_j c_{3k-1-j}$ **return** $(\tilde{P}^{(0)} + \tau \tilde{P}^{(1)}) + B^k (\tilde{P}^{(2)} - \tau \tilde{P}^{(1)})$ *Proof.* Let

$$a^{(0)} = \langle a_0, \dots, a_{2k-2} \rangle, \quad b^{(0)} = \langle b_0, \dots, b_{k-1} \rangle, \quad s^{(0)} = \langle s_0, \dots, s_{2k-2} \rangle,$$

$$a^{(1)} = \langle a_k, \dots, a_{3k-2} \rangle, \quad b^{(1)} = \langle b_k, \dots, b_{2k-1} \rangle, \quad s^{(1)} = \langle s_k, \dots, s_{3k-2} \rangle,$$

$$a^{(2)} = \langle a_{2k}, \dots, a_{4k-2} \rangle, \quad d^{(0)} = \langle d_0, \dots, d_{k-1} \rangle.$$

We first consider $P^{(0)} = \text{MP}_k(s^{(0)}, b^{(1)})$. By definition of the s_i and c_i , we have $a^{(0)} + a^{(1)} = s^{(0)} + B^{2k-1} c_{2k-1}$. Applying Lemma 1 with $m = 2k-1$, $n = k$, $\sigma = +1$, $u = a^{(0)}$, $v = a^{(1)}$, $w = s^{(0)}$, $x = b^{(1)}$, $\kappa_0 = 0$ and $\kappa_i = c_i$ for $1 \leq i \leq 2k-1$, we obtain

$$\begin{aligned} \text{MP}_k(s^{(0)}, b^{(1)}) &= \text{MP}_k(a^{(0)}, b^{(1)}) + \text{MP}_k(a^{(1)}, b^{(1)}) \\ &\quad + \sum_{j=0}^{k-2} b_{k+j} c_{k-1-j} - B^k \sum_{j=0}^{k-1} b_{k+j} c_{2k-1-j}. \end{aligned}$$

Thus $\tilde{P}^{(0)} = \text{MP}_k(a^{(0)}, b^{(1)}) + \text{MP}_k(a^{(1)}, b^{(1)})$.

We handle $P^{(2)} = \text{MP}_k(s^{(1)}, b^{(0)})$ similarly. We have $a^{(1)} + a^{(2)} + c_k = s^{(1)} + B^{2k-1} c_{3k-1}$. Applying Lemma 1 with $m = 2k-1$, $n = k$, $\sigma = +1$, $u = a^{(1)}$, $v = a^{(2)}$, $w = s^{(1)}$, $x = b^{(0)}$ and $\kappa_i = c_{k+i}$ for $0 \leq i \leq 2k-1$ yields

$$\begin{aligned} \text{MP}_k(s^{(1)}, b^{(0)}) &= \text{MP}_k(a^{(1)}, b^{(0)}) + \text{MP}_k(a^{(2)}, b^{(0)}) \\ &\quad + \sum_{j=0}^{k-1} b_j c_{2k-1-j} - B^k \sum_{j=0}^{k-1} b_j c_{3k-1-j}. \end{aligned}$$

Thus $\tilde{P}^{(2)} = \text{MP}_k(a^{(1)}, b^{(0)}) + \text{MP}_k(a^{(2)}, b^{(0)})$.

Finally, for $P^{(1)} = \text{MP}_k(a^{(1)}, d^{(0)})$ we have $d^{(0)} = \tau(b^{(0)} - b^{(1)}) \geq 0$ (the outgoing borrow c'_k is zero, as remarked before the proof). Applying Lemma 2 with $m = 2k-1$, $n = k$, $\sigma = -1$, $u = b^{(0)}$ and $v = b^{(1)}$ if $\tau = +1$ or $u = b^{(1)}$ and $v = b^{(0)}$ if

$\tau = -1$, $w = d^{(0)}$, $x = a^{(1)}$, $\kappa_0 = 0$ and $\kappa_i = c'_i$ for $1 \leq i \leq k$, we obtain

$$\begin{aligned} \text{MP}_k(a^{(1)}, d^{(0)}) &= \tau(\text{MP}_k(a^{(1)}, b^{(0)}) - \text{MP}_k(a^{(1)}, b^{(1)})) \\ &\quad - \sum_{j=0}^{k-1} a_{k+j} c'_{k-1-j} + B^k \sum_{j=k-1}^{2k-2} a_{k+j} c'_{2k-1-j}. \end{aligned}$$

Making the substitution $j \mapsto j + k$ in the last sum, and using $c'_0 = c'_k = 0$, we obtain $\tilde{P}^{(1)} = \tau(\text{MP}_k(a^{(1)}, b^{(0)}) - \text{MP}_k(a^{(1)}, b^{(1)}))$.

The return value is therefore

$$\left(\text{MP}_k(a^{(0)}, b^{(1)}) + \text{MP}_k(a^{(1)}, b^{(0)}) \right) + B^k \left(\text{MP}_k(a^{(1)}, b^{(1)}) + \text{MP}_k(a^{(2)}, b^{(0)}) \right),$$

which is equal to $\text{MP}_{2k}(a, b)$, just as in the polynomial case (see Figure 2). $\square$

3. AN IMPLEMENTATION

In this section we describe an implementation of the Karatsuba integer middle product algorithm in version 4.3.1 of the GMP arbitrary-precision integer arithmetic library [3], and examine its performance. The timing data was obtained on a 2.6 GHz AMD Opteron server running Linux, hosted by the Department of Mathematics at Harvard University.

We first make some remarks on the design of GMP. The lowest ‘mpn’ layer of the library operates directly on natural numbers represented as arrays of digits, each digit occupying a full machine word, so that for example $B = 2^{64}$ on a 64-bit machine. The mpn layer provides an interface for basic operations on such arrays, such as multiple-precision addition, subtraction and multiplication. A portable C implementation is available for each routine, and hand-written assembly versions are supplied for the most important routines for many common processors. GMP also has a higher ‘mpz’ layer that adds memory management, signed arithmetic and other higher-level functionality on top of the mpn layer.

For efficiency, and to make a meaningful comparison with the multiplication code in GMP, our implementation of the middle product is written at the mpn level. We wrote portable C implementations for all routines, and for some routines we also wrote an assembly implementation, targeted at the AMD K8 (Opteron) chip, a 64-bit processor in the x86 family. The code is available from the author’s web page under a free software license.

For small multiplications, GMP implements the quadratic-time classical multiplication algorithm. The K8 assembly implementation in GMP 4.3.1 runs at 2.375 cycles per digit; that is, the speed of the inner loop is such that the cost of an $n \times n$ multiplication is $2.375n^2 + O(n)$ cycles, assuming that all memory accesses hit the L1 cache. This is quite fast, as the 64-bit multiply instruction on the K8 has a maximum throughput of two cycles per multiplication — the additions, carry handling and loop overhead are almost completely hidden behind the multiplication latency. Using the same principles, we wrote an assembly implementation of the quadratic-time classical middle product algorithm, also running at 2.375 cycles per digit. As shown in Figure 3, it runs at essentially the same speed as the ordinary multiplication. Note that both curves have a small jump at $n \approx 37$; we do not know the exact cause, but we suspect that it is related to the K8 branch prediction hardware.

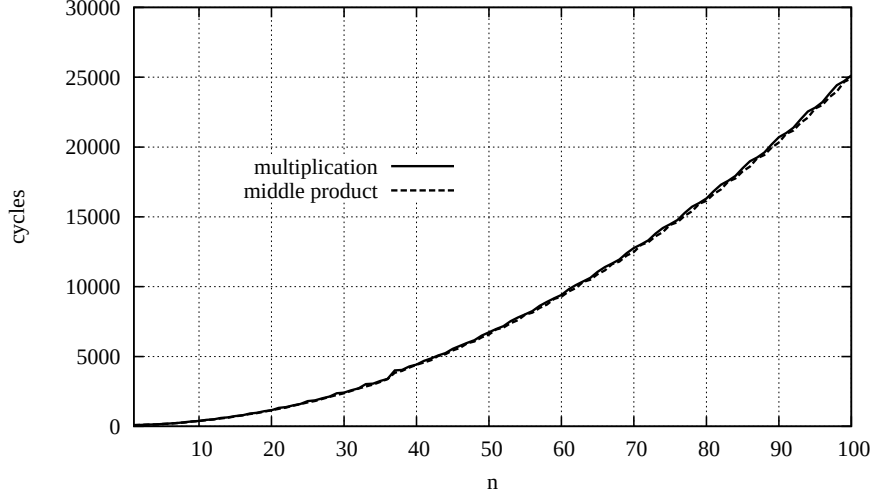


FIGURE 3. Performance of classical multiplication and middle product

For n beyond a certain threshold, GMP switches to the Karatsuba multiplication algorithm, which is called recursively until the problem is small enough that the classical algorithm is more efficient. The threshold depends on the CPU; for the K8 chip it occurs at $n = 28$ digits. The Karatsuba routine is written in C, but the underlying multiple-precision addition and subtraction subroutines have assembly implementations running at 1.5 cycles per digit on the K8.

We now discuss our implementation of the Karatsuba middle product, filling in some details omitted in the statement of Algorithm 2.

First, if n is odd, we reduce to the even n case by handling the contribution from the last row and diagonal of the middle product separately. It should be possible to instead write an unbalanced version that incorporates the extra digits directly into the calculations (as is done by GMP for the ordinary Karatsuba multiplication), but we have not investigated this.

Second, we do not actually store the internal carries $c_1, \dots, c_{3k-1}$ or $c'_1, \dots, c'_k$ anywhere. Instead, we compute the associated error terms during the computation of the corresponding addition (or subtraction) that is generating the carries. For this purpose we introduce a routine $\text{ADDERR}(u, v, \kappa_0, x)$ that takes as input $u = \langle u_0, \dots, u_{n-1} \rangle$, $v = \langle v_0, \dots, v_{n-1} \rangle$, $\kappa_0 \in \{0, 1\}$ and $x = \langle x_0, \dots, x_{n-1} \rangle$, and returns w , κ_n , and the auxiliary sum $\sum_{i=1}^n x_{n-i} \kappa_i$, where $w, (\kappa_1, \dots, \kappa_n) = \text{ADD}(u, v, \kappa_0)$ (similarly for SUBERR and SUB). We wrote an assembly implementation of ADDERR that runs at 3.166 cycles per digit on the K8, meaning that the error term computation costs $3.166 - 1.5 = 1.666$ cycles per digit on top of the cost of the addition itself. Moreover, in two places in Algorithm 2 the same vector of carries is used twice; namely, the two error terms for $\tilde{P}^{(1)}$ use the same carries, and the second error term for $\tilde{P}^{(0)}$ uses the same carries as the first error term for $\tilde{P}^{(2)}$. To take advantage of this redundancy, we also introduce a routine $\text{ADDERR2}(u, v, \kappa_0, x, x')$, identical to ADDERR , except that it computes *two* auxiliary sums $\sum_{i=1}^n x_{n-i} \kappa_i$ and $\sum_{i=1}^n x'_{n-i} \kappa_i$. Our assembly implementation of

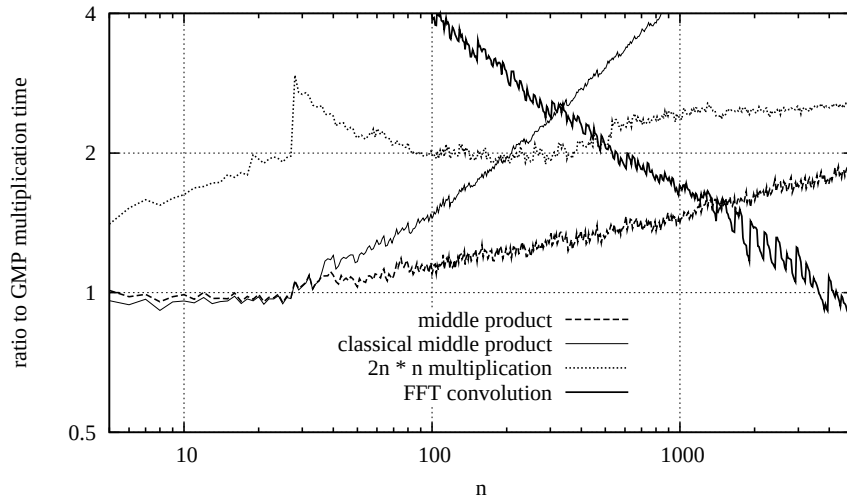


FIGURE 4. Performance of the Karatsuba middle product

ADDERR2 runs at 4.5 cycles per digit on the K8, meaning that these errors terms each cost $(4.5 - 1.5)/2 = 1.5$ cycles per digit over and above the cost of the addition.

Third, working at the level of individual digits entails extra work to keep track of signs and carry propagations. For example, the $P^{(i)}$ each occupy $k + 2$ digits, assuming that $k \ll B$, and the error terms each occupy two digits. The $\tilde{P}^{(i)}$ thus occupy $k + 2$ digits, and might turn out to be negative (we store them in two's complement), and there are two digits of overlap when computing the sum of $\tilde{P}^{(0)} + \tau \tilde{P}^{(1)}$ and $B^k(\tilde{P}^{(2)} - \tau \tilde{P}^{(1)})$ in the last line.

Using the classical and Karatsuba middle product algorithms described above as subroutines, we implemented a general middle product routine that computes $\text{MP}_n(a, b)$. For small n it uses the classical algorithm, switching to Karatsuba for n beyond a certain threshold. On the K8 chip, the optimal threshold is $n = 36$; as expected, this is slightly higher than the Karatsuba threshold for ordinary multiplication, due to the extra linear overhead incurred in computing the error terms.

Figure 4 compares the performance of the middle product routine to three competing algorithms, for $5 \leq n \leq 5000$ digits. The running times for each algorithm are normalised by dividing by the running time for the ordinary $n \times n$ multiplication, using GMP's `mpz_mul_n` routine.

The middle product matches the speed of the ordinary product in the classical region, up to $n = 28$. At this point the ordinary product becomes slightly faster, as it takes advantage of Karatsuba multiplication. Up to $n = 100$ the Karatsuba middle product is no more than 15% slower than ordinary multiplication. At $n \approx 100$, the ordinary multiplication switches to the asymptotically faster Toom-3 algorithm, and then at $n \approx 400$ to the Toom-4 algorithm; these better asymptotics result in the increasingly poor relative performance of the middle product shown in the graph as n increases. (GMP also implements the Schönhage–Strassen integer multiplication algorithm, but this is not relevant for the range of n shown in the graph.)

The second curve in Figure 4 shows the time required for the classical middle product. At $n = 350$, the Karatsuba middle product is already twice as fast as the classical algorithm, and as expected the advantage increases with size.

The third curve shows the time for a $2n \times n$ multiplication, using GMP’s `mpn_mul` routine. The middle product $MP_n(a, b)$ may be extracted from the middle third of the result of such a multiplication (we ignore the $O(n)$ overhead needed to correct for the extra cross-products along the diagonals). Our middle product routine easily outperforms this approach over the range of n shown in the graph. However, we should point out that the performance of unbalanced integer multiplication in GMP 4.3.1 leaves much to be desired. On theoretical grounds, one expects the ratio of the running times of the $2n \times n$ and $n \times n$ multiplications to be approximately 2 for small n and 1.5 for very large n (using FFT-based multiplication algorithms), interpolating reasonably smoothly in between. This is clearly not the case for the data shown in the graph. The GMP authors have indicated that unbalanced multiplication is expected to be greatly improved in a forthcoming GMP release, and we hope to be able to make a fairer comparison then. In the meantime we note that even if the theoretically expected curve is attained, our middle product routine should still be superior for perhaps $n \leq 1000$.

The fourth curve shows the time for a multiplication modulo the Fermat modulus $B^{2n} + 1$, using GMP’s internal `mpn_mul_fft` routine, a subroutine of its Schönhage–Strassen multiplication code (we used the improved version described in [2]). The middle product $MP_n(a, b)$ may be recovered from this operation, since the highest third of the output ‘wraps around’ to overlap the lowest third, but does not interfere with the middle third. As shown in the graph, this approach begins to outperform our middle product routine at about $n \approx 1500$.

In summary, we see that the new middle product routine performs very favourably for $n \leq 1000$, compared to the obvious alternatives.

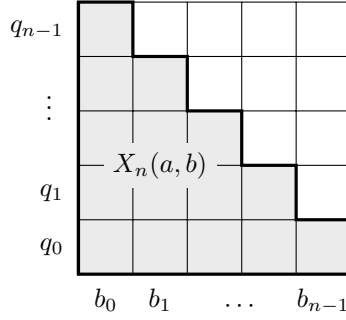
4. APPLICATIONS TO INTEGER DIVISION

In this section we consider an application of the middle product to the problem of B -adic division. We are given integers $a = \langle a_0, \dots, a_{n-1} \rangle$ and $b = \langle b_0, \dots, b_{n-1} \rangle$, and wish to compute $q = a/b \bmod B^n$. We assume that B is a power of two, that b_0 is odd and that $n \ll B$. This operation has applications to problems such as exact integer division [6] and computing greatest common divisors [8]. It should be possible to handle the more frequently used truncating (left-to-right) division using similar techniques; we chose to illustrate with B -adic division because it is technically simpler, avoiding the issue of quotient digit estimation.

Our algorithm is based on an analogous recursive divide-and-conquer algorithm for polynomials given in [5, Algorithm MP-divide, p. 422], with several modifications to account for carries. To make the recursion go through, the algorithm needs to return slightly more information than just q ; it must also return a quantity $w = W_n(a, b)$ that we call the *overflow*, defined as follows. Let $q = a/b \bmod B^n$, and consider the sum

$$X_n(a, b) = \sum_{0 \leq i+j < n} b_i q_j B^{i+j},$$

which incorporates the digit-by-digit products from the region shown in Figure 5. In particular, all the products contributing to the result modulo B^n are included,

FIGURE 5. Digit products contributing to $X_n(a, b)$

so we have $X_n(a, b) \equiv bq \equiv a \pmod{B^n}$. We put

$$W_n(a, b) = \frac{X_n(a, b) - a}{B^n}.$$

Note that $W_n(a, b)$ is an integer, and if $n \ll B$ then $0 \leq W_n(a, b) < B^2$.

We put $\text{BDIV}_n(a, b) := (q, w)$, where q and w are defined as above. For small n , the most efficient way to compute $\text{BDIV}_n(a, b)$ is by a classical quadratic-time division algorithm: first compute $q_0 = b_0^{-1}a_0 \pmod{B}$, subtract q_0b from a , and repeat, keeping track of contributions to w . We assume that the classical algorithm is provided as a subroutine. Algorithm 3 below gives a recursive algorithm based on the middle product. When n falls below a certain threshold, the recursive BDIV calls should switch to using the classical algorithm.

Algorithm 3: Divide-and-conquer B -adic division with overflow

Input: integers $a = \langle a_0, \dots, a_{n-1} \rangle$ and $b = \langle b_0, \dots, b_{n-1} \rangle$ with $2 \leq n < B$

Output: $(q, w) = \text{BDIV}_n(a, b)$

$s \leftarrow \lceil n/2 \rceil, \quad t \leftarrow \lfloor n/2 \rfloor \quad (= n - s)$

$a^{(0)} \leftarrow \langle a_0, \dots, a_{s-1} \rangle, \quad a^{(1)} \leftarrow \langle a_s, \dots, a_{n-1} \rangle$

$q^{(0)}, w^{(0)} \leftarrow \text{BDIV}_s(a^{(0)}, \langle b_0, \dots, b_{s-1} \rangle)$

$u = \langle u_0, \dots, u_{t+1} \rangle \leftarrow w^{(0)} + \text{MP}_{n-1,s}(\langle b_1, \dots, b_{n-1} \rangle, q^{(0)})$

$a' \leftarrow a^{(1)} - \langle u_0, \dots, u_{t-1} \rangle \pmod{B^t}$

if $a^{(1)} < \langle u_0, \dots, u_{t-1} \rangle$ **then** $c \leftarrow 1$ **else** $c \leftarrow 0$

$q^{(1)}, w^{(1)} \leftarrow \text{BDIV}_t(a', \langle b_0, \dots, b_{t-1} \rangle)$

$q \leftarrow q^{(0)} + B^s q^{(1)}$

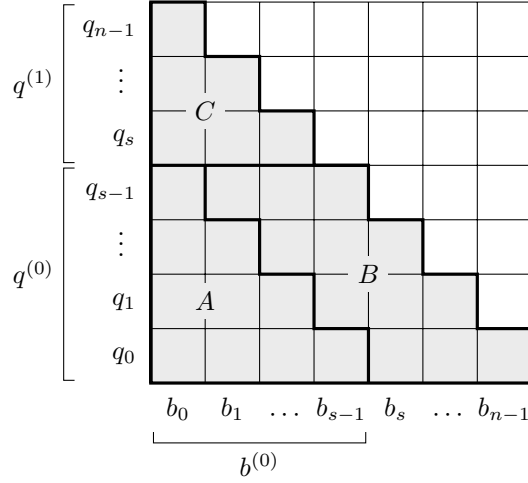
$w \leftarrow w^{(1)} + \langle u_t, u_{t+1} \rangle + c$

return q, w

Proposition 4. *Algorithm 3 correctly computes $\text{BDIV}_n(a, b)$.*

Proof. After the first BDIV call we have $bq^{(0)} \equiv a^{(0)} \pmod{B^s}$ and

$$a^{(0)} + B^s w^{(0)} = \sum_{j=0}^{s-1} \sum_{i=0}^{s-1-j} b_i q_j B^{i+j};$$

FIGURE 6. Divide-and-conquer B -adic division (Algorithm 3)

this sum corresponds to region A in Figure 6. The middle product (region B) computes

$$\text{MP}_{n-1,s}(\langle b_1, \dots, b_{n-1} \rangle, q^{(0)}) = \sum_{j=0}^{s-1} \sum_{i=s-j}^{n-1-j} b_i q_j B^{i+j-s}.$$

(Note that this middle product is balanced if n is even, and almost balanced if n is odd.) Thus we have

$$B^s u = -a^{(0)} + \sum_{j=0}^{s-1} \sum_{i=0}^{n-1-j} b_i q_j^{(0)} B^{i+j}.$$

The definitions of a' and c imply that

$$a' - cB^t = a^{(1)} - \langle u_0, \dots, u_{t-1} \rangle = a^{(1)} - u + B^t u'$$

where $u' = \langle u_t, u_{t+1} \rangle$. After the second BDiv call (region C), we have $bq^{(1)} \equiv a' \bmod B^t$ and

$$a' + B^t w^{(1)} = \sum_{j=0}^{t-1} \sum_{i=0}^{t-1-j} b_i q_{j+s} B^{i+j}.$$

Multiplying by B^s and adding to the earlier sum we obtain

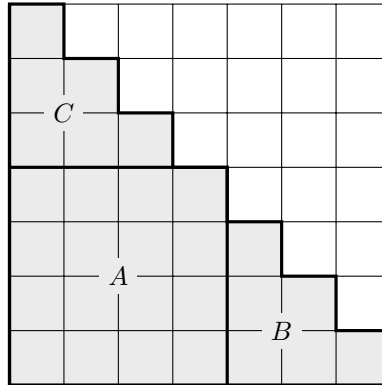
$$a^{(0)} + B^s(a' + u) + B^n w^{(1)} = \sum_{j=0}^{n-1} \sum_{i=0}^{n-1-j} b_i q_j B^{i+j} = X_n(a, b).$$

The left hand side is equal to

$$a^{(0)} + B^s(a^{(1)} + cB^t + B^t u') + B^n w^{(1)} = a + B^n(w^{(1)} + u' + c).$$

This shows that the returned quotient q and overflow w are correct. $\square$

We now consider the performance of an implementation of Algorithm 3 in GMP, building on the implementation of the Karatsuba middle product discussed in the previous section.

FIGURE 7. GMP's divide-and-conquer B -adic division

GMP 4.3.1 contains several routines for B -adic division: a classical quadratic-time algorithm for small n , a divide-and-conquer algorithm for inputs of moderate size, and several other algorithms intended for much larger operands that we do not consider here. We wrote our own implementation of the classical algorithm, as the version supplied with GMP does not compute the overflow term. The two implementations run at essentially the same speed, differing only by one or two percent.

The divide-and-conquer algorithm in GMP is different to Algorithm 3, and in particular does not use the middle product. Briefly, it runs as follows. First, it computes the low half of the quotient *and* an associated remainder (region A in Figure 7). This operation itself uses a divide-and-conquer approach, essentially the Burnikel–Ziegler algorithm, switching to classical division-with-remainder for small enough n . Second, it computes the truncated product corresponding to region B, and subtracts this from the dividend. Finally, it recursively computes the second half of the quotient (region C).

Figure 8 compares our implementations of the classical algorithm and the divide-and-conquer algorithm (Algorithm 3) for $5 \leq n \leq 50$. The threshold for switching algorithms is $n = 22$. Note that this is *lower* than the Karatsuba middle product threshold ($n = 36$); indeed, the fact that the divide-and-conquer division is faster than the classical division for $22 \leq n \leq 36$ has nothing to do with the Karatsuba middle product. Rather, it occurs because the classical middle product runs at 2.375 cycles per digit (as discussed in the previous section), whereas the classical division algorithm runs at only 2.5 cycles per digit; the former processes two rows of the product at a time, making better use of CPU registers. It should be possible to make the latter also run at the same speed, but we have not implemented this. We would then expect the threshold between classical and divide-and-conquer division to increase to a little beyond $n = 36$.

Figure 9 shows the performance of Algorithm 3 relative to the existing B -adic division routines in GMP. We note several salient features. For $n < 22$, as mentioned above, the divide-and-conquer algorithm is slower than GMP's classical division. Next, for n up to about 200, the divide-and-conquer division is about 10% faster than GMP's division. This improvement is mainly due to the efficiency gained via the Karatsuba middle product. In fact, GMP's divide-and-conquer division

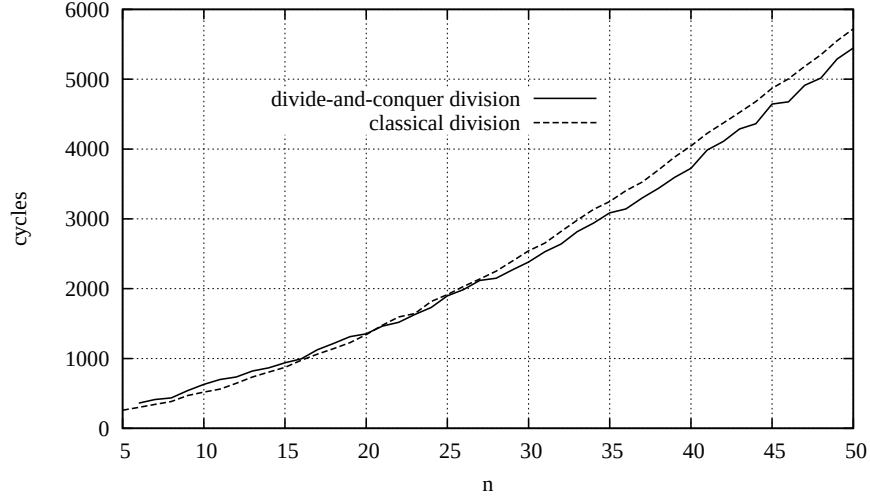


FIGURE 8. Performance of classical and divide-and-conquer B -adic division

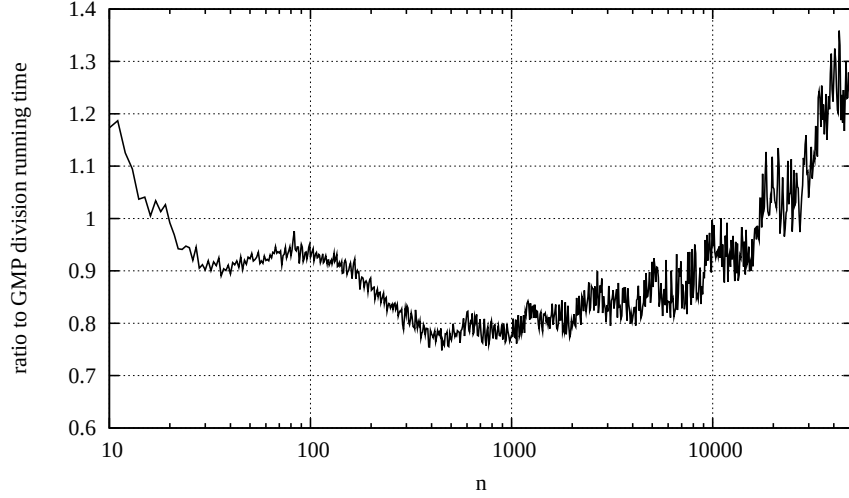


FIGURE 9. Performance of divide-and-conquer B -adic division

only begins to beat classical division around $n = 280$. The main bottleneck in GMP's divide-and-conquer algorithm is the use of the truncated product, which becomes progressively less efficient as n increases; for large n the truncated product takes the same time as a full product (see [7] for further discussion). This effect becomes more pronounced as n increases, and we see the advantage enjoyed by Algorithm 3 widening to around 20% up to about $n = 2000$. At this stage GMP's divide-and-conquer begins to catch up, breaking even at about $n = 20000$, and then pulling ahead beyond this point. This reversal occurs because the truncated product is able to take advantage of the Toom-3 (and higher order) multiplication

algorithms. These are *asymptotically* faster than the middle product, which has at best Karatsuba complexity.

ACKNOWLEDGEMENTS

Many thanks to Torbjörn Granlund and Paul Zimmermann for their comments and suggestions, and to Torbjörn Granlund for his invaluable assembly programming assistance. Thanks to the Mathematics Department at Harvard University for providing the hardware on which the profiles were performed.

REFERENCES

1. A. Bostan, G. Lecerf, and É. Schost, *Tellegen's principle into practice*, Symbolic and Algebraic Computation (J. R. Sendra, ed.), ACM Press, 2003, Proceedings of ISSAC'03, Philadelphia, August 2003., pp. 37–44.
2. Pierrick Gaudry, Alexander Kruppa, and Paul Zimmermann, *A GMP-based implementation of Schönhage-Strassen's large integer multiplication algorithm*, ISSAC 2007, ACM, New York, 2007, pp. 167–174.
3. Torbjörn Granlund, *The GNU Multiple Precision Arithmetic library*, <http://gmplib.org/>.
4. Guillaume Hanrot, Michel Quercia, and Paul Zimmermann, *Speeding up the division and square root of power series*, Research Report 3973, INRIA, July 2000.
5. ———, *The middle product algorithm, I.*, Appl. Algebra Engrg. Comm. Comput. **14** (2004), no. 6, 415–438.
6. Tudor Jebelean, *An algorithm for exact division*, J. Symbolic Comput. **15** (1993), no. 2, 169–180.
7. Thom Mulders, *On short multiplications and divisions*, Appl. Algebra Engrg. Comm. Comput. **11** (2000), no. 1, 69–88.
8. Damien Stehlé and Paul Zimmermann, *A binary recursive gcd algorithm*, Algorithmic number theory, Lecture Notes in Comput. Sci., vol. 3076, Springer, Berlin, 2004, pp. 411–425.